

Naming consistency for Vantage framework elements ✓

my framework is called vantage, and one of the element in the framework is called vista. do you think they are on-theme ?

Yes, they're well on-theme — both come from the same semantic field of *elevated viewpoints* and *seeing far*.

- **Vantage:** a position giving a strategic or commanding view (as in "vantage point")
- **Vista:** a long, sweeping view, especially one seen through or along an avenue

The metaphor fits a data abstraction framework rather nicely too:

- *Vantage* as the framework = the elevated platform you stand on to survey your data landscape
- *Vista* as an element = a specific view/perspective onto that landscape

It's coherent without being on-the-nose. A few related words in case you ever want to extend the naming scheme for other components: **panorama**, **horizon**, **outlook**, **prospect** (in its older sense of "a view"), **scope**, **lens**, **aperture**, **observatory**, **summit**, **overlook**.

One small thing worth being aware of: there's a reasonably well-known company called Vantage (cloud cost observability) and "Vista" is a fairly loaded word (Windows Vista, Vista Equity Partners, plus countless smaller products). For an internal/personal Rust crate it doesn't matter, but if you ever publish to crates.io the names might already be taken or create SEO collisions. Worth a quick check.

now - vista in a framework is abstraction over data tables. It's like a window into wide range of diverse table implementations.

however all tables are static and have limited capabilities. CSV table , for example, is read-only. DynamoDB table cannot sort and some APIs are limited on pagination and conditions.

I need to introduce new concept, which wraps Vista and adds extra

Show more

Great question — and there's a real semantic distinction to nail here, not just a vibe match. What you're describing isn't *another view*; it's a **composed/enhanced view that adds capability**. The name should signal that layering or augmentation, while staying in the scenery family.

Here are the strongest candidates, grouped by what they emphasize:

Composition / layering (my top picks)

- **Overlook** — literally an elevated structure *built* to enhance a view. Implies something constructed on top. "An overlook composes multiple vistas into a richer perspective." Reads beautifully in code: `Overlook::from(read_vista, write_vista)`.
- **Panorama** — a wider, composed view stitched from multiple vantage points. Strongly implies *combination* of views, which matches your read+write fusion exactly. `Panorama::compose(...)`.
- **Prospect** — in its archaic sense, a commanding/extended view, often *designed* (think landscape architecture: "a prospect was cut through the trees"). Slightly more obscure but elegant.

Augmentation / framing

- **Lens** — a vista seen *through* something that enhances it. Familiar to Rust/FP folks (van Laarhoven lenses) which is either a feature or a footgun depending on whether you want that association. Probably a footgun — composable read/write lenses in FP mean something specific and your construct isn't that.
- **Scope** — instrument metaphor (telescope, periscope), implies enhanced reach. But "scope" has heavy meaning in programming already (variable scope, scoped types) — likely too overloaded.
- **Aperture** — an opening that controls what passes through. Nice metaphor for "this construct gates and shapes what the underlying table can do." But less obviously *scenic*.

Composed landscape

- **Vantage** — wait, you already used this for the framework. Skip.
- **Outlook** — fine but bland and Microsoft-coded.
- **Horizon** — too vague, doesn't suggest composition.

My recommendation: **Overlook**.

Reasons it wins over Panorama (the runner-up):

1. The verb form is built in — *to overlook* something means to survey it from above, which is exactly what your construct does to the wrapped vistas.
2. It's clearly a *constructed* thing (you build an overlook on a hill), which matches "wraps a vista and adds capabilities."
3. Shorter, less pretentious in type signatures: `Overlook<T>` reads cleaner than `Panorama<T>`.
4. There's a minor pun-bonus: "overlook" can also mean "miss something," which is mildly amusing given the construct papers over the *missing* capabilities of the underlying table. (Optional: lean into this in docs, or ignore it.)

One caveat to think about: if you anticipate *more* layers later (e.g. an overlook composed of overlooks, or different flavors of enhancement — caching, transactions, etc.), you might want a more generic term and reserve `Overlook` for the specific read+write fusion case. In that case the hierarchy could be:

```
Vista          // base abstraction over a table
Overlook       // read+write fusion specifically
Panorama       // generic composed/enhanced vista
```

Or invert it depending on which is the more common case. But if there's really just one tier of wrapping, `Overlook` alone does the job.

i think we should reinforce the ability to enhance and combine.

If enhancement and combination are the load-bearing ideas, **Overlook** loses some of its appeal — it implies *position* (built on top, surveying) more than *fusion*. You want a word that says "multiple things become one richer thing."

Here are the candidates re-ranked through that lens:

Strongest fits

- **Panorama** — the definitional meaning is a *stitched-together wide view from multiple perspectives*. Etymologically Greek "pan" (all) + "horama" (view) = "see all." It's almost literally "compose vistas into something bigger." Reads well: `Panorama::compose(read_vista, write_vista)`, `Panorama::stitch(...)`. The verb *to panoramize* doesn't really exist, but *compose* / *stitch* / *assemble* all feel natural with it.
- **Tableau** — a *composed scene*, often deliberately arranged. Comes from French for "picture/scene" but in English carries the connotation of carefully assembled

tableau vivant. Bonus: it literally contains "table," which is a delightful pun for a data-table framework. Downside: Tableau Software exists and is data-adjacent, so it might create confusion (though it's a different layer of the stack).

- **Diorama** — a constructed scene combining multiple elements into a unified whole. Strongly implies *built from parts*. More playful/whimsical than Panorama. Slight downside: dioramas are usually static, which contradicts the "enhanced capability" angle.

Worth considering

- **Montage** — explicitly about combination. But it's cinematic rather than scenic, breaking the theme.
- **Composite** — accurate but generic and un-scenic; sounds like a base class in a Java textbook.
- **Mosaic** — combination of pieces into a whole. Scenic-adjacent (mosaics often depict scenes) but more about *tiling* than *viewing*.

My recommendation: Panorama.

It's the cleanest semantic match for "combine vistas into something with broader reach and more capability." The metaphor is exact: a panorama is what you get when you stitch multiple views together, and the result genuinely shows you more than any single view could. Code reads naturally:

```
rust
let panorama = Panorama::new()
    .with_reader(csv_vista)
    .with_writer(dynamo_vista)
    .with_sort(in_memory_sort)
    .build();
```

If you want the runner-up: **Tableau**, purely for the "contains 'table'" pun and the "deliberately composed scene" connotation. It's the more playful choice; Panorama is the more obvious/communicative one.

One naming-architecture thought: you now have a three-word family — **Vantage** (framework), **Vista** (single view), **Panorama** (composed/enhanced view). That's a tight, memorable trio and there's natural room to grow (Horizon, Overlook, Prospect, Lens) if other concepts emerge later. I'd lock in Panorama and move on.

i like diorama, especially since it can beshortened into Dio.

Diorama is a great call, and **Dio** as a short form is genuinely lovely — three letters, easy to type, pleasant to say, and it gives you a natural pair: **Vista / Dio**.

The "constructed scene from multiple elements" metaphor maps really well onto what you're building:

- A diorama is **assembled** from disparate pieces (different table backends, different capabilities)
- It presents them as a **unified scene** (one coherent API surface)
- The whole is **richer than any single piece** (read+write, sort-over-unsortable, etc.)
- You can **arrange and compose** the elements deliberately

The one concern I raised earlier — that dioramas are static — actually dissolves on reflection. The diorama itself is static as a *presented artifact*, but it's built from parts that each do their own thing. In your case, Dio is the composed surface; the underlying vistas still do the live work. That fits.

A few practical notes:

Naming in code reads nicely. `Dio<T>`, `dio.read()`, `dio.write()`, `Dio::compose(...)`, `DioBuilder` — all clean. The full `Diorama` works for type names if you want the formal version with `Dio` as a type alias or shorthand in docs and conversation.

Verb form is awkward. "Dioramize" isn't a thing. You'll want to lean on *compose*, *assemble*, *build*, *arrange* in method names. `Diorama::assemble(read_vista, write_vista)` reads quite well actually — assembling a diorama is exactly what people do.

Watch the Ronnie James Dio collision. Mostly harmless — different domain entirely — but if you ever search "dio rust" you'll get heavy metal results. Not a real problem, just amusing.

The family is now: Vantage / Vista / Dio. That's a strong, distinctive set. Vantage and Vista pair on the V-alliteration; Dio breaks the pattern but earns it by being short and memorable. If anything, the break makes Dio stand out as *the composed thing* — visually different in code from the building blocks it wraps.

Good pick. Ship it.

ok, on a more serious note, entertain this for a bit.

suppose we have a data storage. It's eventual consistency, can return errors, has rate-limit access, can return data that's out of date, has

inflexible interface with limited capabilities - e.g. lack of sort or filtering.

on other hand - our UI must show up-to-date data, sortable, use virtual data source (infinite scroll), pre-fetch data, auto-retry, cache locally for

Show more

This is a genuinely interesting design problem and I want to make sure I understand the shape of it before throwing concepts at you. Let me ask the questions that will most shape the architecture.

1. What's the actual unit of work between backend and the reactive layer?

You said "page" but I want to pin this down. When the backend can't sort or filter, but the frontend can request sorted/filtered views, *who's doing the sorting?* Three possibilities:

a) The reactive layer fetches everything ever and sorts/filters in memory (works for small datasets, not for "all DynamoDB rows") b) The reactive layer fetches *pages in backend's native order* and the frontend only ever sees data in that order (limiting) c) The reactive layer has some hybrid: knows about indices/secondary structures, may fetch multiple "streams" and merge them

The answer here completely changes what the caching layer looks like.

2. What's the consistency contract you want to offer the frontend?

The frontend wants "up-to-date data," but the backend is eventually consistent. So when the user edits a record:

- Optimistic: frontend sees its own write immediately, even before backend confirms
- Read-your-writes: the reactive layer guarantees the user sees their own edits, even if other users wouldn't yet
- Eventually consistent passthrough: frontend just sees whatever's there, with some staleness indicator

These imply very different caching/invalidation strategies. Pick one or describe the spectrum you want to support.

3. How does the frontend express "what I care about right now"?

You mentioned the rust thread can read frontend state — visible records, scroll depth, etc. Is the model:

a) **Pull**: frontend declares "I want rows 1000-1050 sorted by X" and the reactive layer fulfills that b) **Subscribe**: frontend declares "I'm interested in this query" and the reactive layer maintains a live view that frontend renders from c) **Hint**: frontend says "user is looking at rows ~1000, prefetch hint" and the reactive layer decides what to do (I suspect the answer is "all three at different layers" but I want your view.)

4. What does "user adds a new record" actually mean when the backend is unreliable?

Specifically: does the new record have an ID assigned by the client (UUID) or by the backend (auto-increment)? This dramatically changes what "auto-retry on poor network" looks like — client-assigned IDs let you treat writes as idempotent retries; server-assigned IDs mean you have to track pending writes through a different mechanism.

5. Is the cache a "view materialization" or a "record store"?

Two very different mental models:

a) **Record store**: cache is keyed by record ID. Queries are answered by combining the cache with backend fetches. When a record is edited, you update the cached record and any view that includes it. b) **View materialization**: cache stores the results of specific queries ("first 50 rows sorted by created_at desc"). Edits invalidate or update materialized views.

The first is conceptually cleaner but harder when the backend can't filter/sort. The second is more pragmatic but has nasty invalidation problems.

6. How fresh is "fresh enough"?

The frontend wants up-to-date data. But "up-to-date" against an eventually-consistent backend is a category error. You need some staleness policy:

- TTL per query?
- TTL per record?
- "Refresh visible rows every N seconds"?
- "Refresh on focus / scroll stop / user idle"?
- Background poll + push notifications when supported?

Related: do you want the reactive layer to expose *staleness* to the frontend (so it can show a spinner or "synced 3s ago"), or hide it entirely?

7. Backpressure and cancellation.

User scrolls fast through pages 1→2→3→4→5. Backend is slow. By the time page 2's fetch returns, user is on page 5. What happens?

- Page 2's result still goes into the cache (it's correct data, just not currently visible)

- Page 2's fetch is cancelled mid-flight to free up the rate limit budget for page 5
- Reactive layer reorders the queue, prioritizing page 5

Probably "all of these depending on configuration," but where does that policy live?

8. What's the failure surface to the frontend?

You said "convert fallible backend into infallible reactive data source." But some things genuinely can't succeed (user tries to write to a backend that's been offline for an hour; auth expired; record was deleted by another user; permission denied).

Is "infallible" really *"the data source itself never errors, but individual operations may end up in a permanent failed state that the frontend can inspect"*? Or is there some deeper guarantee?

If I had to guess at the shape of your answers, I'd predict something like:

- The reactive layer is a **per-query subscription** (option 2 + 3 in Q3)
- Cache is a **record store with secondary view indices** (hybrid of Q5)
- Writes use **client-assigned IDs and an outbox pattern** (Q4)
- Consistency is **read-your-writes with optimistic UI** (Q2)
- Staleness is **explicit in the API** so frontends can show indicators (Q6)
- The reactive layer is a **scheduler over capability-typed backend operations** — meaning each driver declares what it can do (sort? filter? batch read? conditional write?) and the scheduler plans around the gaps

The last point connects back to Vista/Diorama nicely: Vista declares capabilities, the reactive layer (something new — maybe **Atelier**? It's where the diorama gets built) orchestrates around the declared gaps.

Tell me which of these eight questions matter most and we'll go deep on those first.

- a) yes something in the middle must do the sorting. In theory we can probably fetch data and re-sort them progressively - updating the UI after each new page has arrived. That would let user see results as soon as they arrive, and potentially cache received data so far - resume if he re-opens the app.
- b) some records in the tabel could be "stale", that means we will display them. but they might be inaccurate. For instance. if user re-opens app on

Show more

Good, this is taking shape. I have a clearer picture now and want to reflect it back, push on a few load-bearing parts, and then ask the next round of questions. I'll try to be concrete rather than hand-wavy.

What I'm hearing

There's a long-lived **session object** (per query/view) that bridges frontend and backend. The frontend sends async events into it ("filter by 'cat'", "refresh", "I'm scrolled to row 1000", "edit row X"), and the session emits a stream of structured updates back ("here are rows 0-49", "3 rows changed", "row 47 removed", "all data invalidated"). The session is responsible for orchestrating the unreliable backend on one side and presenting a coherent reactive view on the other. The cache has two layers: a record store (`id → row`) and a query-index store (`sort+filter → ordered list of ids, possibly chunked`).

That's a clean model. It maps onto something a lot like a **materialized view subscription** — which is well-trodden territory in databases, GraphQL clients (Apollo/Relay), and replication systems, so we can borrow patterns without reinventing them.

Let me name things tentatively so we can talk concretely:

- **Vista** — capability-typed view over one table backend (what you have)
- **Diorama** — composed scene fusing multiple vistas into a richer surface (what we discussed)
- **Session** (placeholder name — *Scene? Tableau? Exhibit?*) — the long-lived subscribed view that the frontend talks to
- **Curator** (placeholder) — the background machinery that fetches, retries, schedules, caches

I'll use these names below; we can rename later. **Scene** feels right for the session — it's what the visitor (frontend) currently observes, and it's the diorama-in-action.

Things I want to push on

1. The two caches are coupled, and that coupling is where most bugs will live.

You described `id → row` and `(sort, filter) → chunked id lists`. The hard cases:

- A row's data changes such that it should no longer match the filter, or should move position in the sort order. The id-list cache is now stale even though the row cache is fresh.
- A new row is added. Every cached query that *might* include it is now stale.
- A row is deleted. Every cached id-list that contains it now has a phantom.

You can handle this three ways, and you should pick consciously:

a) **Pessimistic**: any write invalidates all id-list caches. Simple, correct, but throws away a lot of work. b) **Optimistic with reconciliation**: on write, update the row cache and patch the id-list caches based on local knowledge ("this row now matches filter X, insert at position Y based on sort"). Then verify against backend later. Fast but risks divergence. c) **Server-authoritative invalidation**: writes go to backend, backend (or your wrapper) tells you which queries are affected, you invalidate those. Requires backend cooperation or a smart middle layer.

For an unreliable backend, (a) is too wasteful, (c) is what you actually want but is the hardest, (b) is the pragmatic middle ground. I'd offer (a) as the default that drivers get for free, and let drivers opt into (b) or (c) with hooks.

2. "Chunked id lists" — what's the chunk boundary?

This matters a lot. Two options:

- **Backend-page-aligned**: chunks correspond to backend pagination cursors. Easy to refresh ("re-fetch chunk 3"), but the chunk size is whatever the backend gives you, and chunks shift when rows are inserted/deleted.
- **Frontend-window-aligned**: chunks correspond to fixed offsets (rows 0-99, 100-199...). Stable for the UI's virtual scroller, but a single insert at row 0 shifts everything.

In practice I think you want **backend-aligned with a translation layer**. The cache stores backend-native chunks; the frontend window resolves "rows 1000-1050" by walking the chunk index. Inserts/deletes update the chunk index but don't require recomputing offsets.

For sorted/filtered queries where the backend can't sort/filter, the "chunks" are really logical — you've fetched some superset, sorted it locally, and produced a virtual ordering. That's a different beast and probably wants its own representation.

3. Stale records need a status field, and so do queries.

You mentioned visual cues for stale rows and disabling edits on them. That implies every row the frontend sees has metadata, not just data:

```
RecordView {  
  data: Row,  
  status: Fresh | Stale { fetched_at } | PendingWrite | WriteFailed { error },  
  capabilities: { can_edit: bool, can_delete: bool, ... }  
}
```

Same at the query level:

```
SceneStatus {
  loading: bool,          // fetching first page
  refreshing: bool,       // user-triggered refresh in flight
  complete: bool,         // all data fetched, no more pages
  stale: bool,            // data is from before the connection drop
  error: Option<Error>,    // last error encountered (non-fatal)
}
```

The frontend renders both. This is the "infallible interface that exposes failure as data" pattern — operations don't return `Result`, they emit state.

4. The event/message vocabulary needs to be small and total.

You sketched: refresh request, "no changes", "N rows changed", "all data changed", "row removed", "row added". Let me try to compress this into a minimal sufficient set.

From frontend to Scene (commands):

- `Subscribe(query)` — start a view
- `SetViewport(range)` — what the user can see (drives prefetch + priority)
- `Refresh` — user pulled to refresh
- `Edit(id, patch)` / `Insert(record)` / `Delete(id)` — mutations
- `Unsubscribe` — close the scene

From Scene to frontend (updates):

- `Replace(range, rows)` — these rows are now this (covers initial load, refresh-with-changes, range-fetch-completed)
- `Patch(id, new_data)` — single row changed
- `Insert(position, row)` / `Remove(id)` — structural change
- `StatusChanged(SceneStatus)` — metadata, not data
- `Invalidated` — throw everything away, I'm about to send you a fresh world

That's six commands and five updates. Almost everything else (e.g. "3 rows changed") is expressible as a `Patch` per row or a `Replace` over a range.

The hard call: is `Patch` distinct from `Replace(single_row)`? I think yes, because patches preserve frontend state like row selection/animation, while replaces are conceptually "this is new data, treat it accordingly."

5. Progressive sort while fetching is gorgeous but has a UX trap.

Showing partial sorted results "as they arrive" works great if the user understands they're partial. It can be deeply confusing if they think they're seeing the final answer and scroll past the row they wanted. The Scene should probably emit progress

(`fetched 340/~2000`) as part of status, and the frontend can show "Sorting... 17% complete" or a shaded incomplete region.

This also interacts with the cache: do you cache the partial sorted result? Probably not — cache the raw chunks, recompute the merge each time. Cheaper than it sounds because the chunks are already locally ordered.

Questions for you

Q1. When the frontend says "I'm scrolled to row 1000," is that a *hint* (Scene may prefetch around it) or a *demand* (Scene must have those rows ready before responding to anything else)? In other words: does viewport drive scheduling priority, or does it drive the entire fetch plan?

Q2. Are scenes independent or shared? If two scenes have overlapping queries (e.g. one is "all cats" and another is "cats sorted by age"), do they share the row cache? Share fetched chunks? I assume row cache is shared (a row is a row), but query results are per-scene. Confirm?

Q3. Writes: when a user edits a row in scene A, how does scene B (showing the same row) learn about it? Three options:

- Scene A applies the patch optimistically, writes to backend, on success the row cache is updated, all scenes subscribing to that row id get a Patch event.
- Same as above but optimistic update flows through the row cache immediately.
- Scene A is the only one that knows until backend confirms and an external invalidation arrives.

I'd push for option 2 (row cache is a tiny event bus keyed on id; optimistic writes update it instantly and emit; backend confirmation is a no-op if successful and a rollback if not). But this needs your call.

Q4. Mutations during in-flight refresh. User edits row 47. While the optimistic patch is propagating, a refresh completes and brings back the *old* version of row 47 from the backend (because the write hasn't reached backend yet, or backend is eventually consistent). What wins? Common answer: pending writes are a separate layer applied *on top* of the cache, so refresh updates the cache but the pending edit is reapplied on read.

Q5. What's the granularity of "capability"? You mentioned read-only CSV, unsortable DynamoDB, limited pagination. Is the capability set fixed and known up front (e.g. an enum `{ Read, Write, Sort, Filter, BatchRead, ConditionalWrite, ... }`), or is it open-ended? Fixed is easier to schedule against; open-ended is more honest about real backends. I'd lean fixed-with-extensions: a known core plus a typed extension bag.

Q6. Driver implementation — what's the *minimum* a driver must implement, and what does the framework provide automatically? My proposed answer:

- **Driver must implement:** how to fetch a single record by id; how to fetch the next page from a cursor; how to perform a write.
- **Framework provides for free:** caching, retry, rate-limit shaping, viewport-driven prefetch, optimistic writes with rollback, progressive in-memory sort/filter when the driver doesn't support them, the entire Scene/event protocol.
- **Driver may opt into:** native sort, native filter, change notifications/subscriptions, batch operations, conditional writes. Each opt-in lets the framework be smarter.

This is the "implement the gaps" model you described. Each capability is a trait the driver does or doesn't implement, and the Curator's scheduler plans differently based on what's available.

If I had to declare the architecture in one paragraph: a **Scene** is a long-lived materialized subscription owned by the frontend; it speaks a small command/update protocol; it's backed by a **Curator** that schedules work against a capability-typed **Diorama** of **Vistas**; the cache is two-layered (row store keyed by id, query index keyed by query parameters) with the row store doubling as an event bus; drivers implement a minimum surface and opt into capabilities to let the Curator be smarter; everything fallible is exposed as status data on rows and scenes rather than as **Result** types at the API boundary.

That feels coherent. Tell me where it's wrong or what to dig into next — I'd particularly like your answers to Q3 and Q4, because writes-with-optimism is where I expect the design to either feel elegant or get tangled.

lets retry but adjust terms

what user sees = scenery.

fallabe backend = vista.

lens = mechanism that works hard to coordinate, fetch, store cache, exposes scenery.

dio = ways for developers to use multiple vistas to satisfy requirements of lens.

in terms of comms - we use rust primitives - channels, signals and async code.

Right, with those terms locked in the architecture re-renders more cleanly. Let me redraw it.

The vocabulary

- **Vista** — one fallible backend table. Speaks its native dialect, has whatever capabilities it has (read-only, no sort, paginated weirdly, rate-limited, eventually consistent).
- **Dio** — composition of multiple vistas to satisfy a richer contract. A read-only CSV vista + a write-capable in-memory vista becomes a read-write dio. A sort-incapable vista + a local sort step becomes a sortable dio. Dios are *recursive composition*: a dio is itself usable where a vista is expected, because it presents the same capability surface (just a richer one).
- **Lens** — the live, persistent coordination object. Owns the cache, the scheduler, the retry policy, the optimistic write log, the subscription machinery. A lens *focuses* a dio (or vista, since $\text{dio} \supseteq \text{vista}$) into a reactive, infallible surface.
- **Scenery** — what the frontend renders. The observable output of a lens. State, not commands. Rows with status, query status, viewport state.

So the picture is: **Vista(s) → Dio → Lens → Scenery**. Composition layer, then coordination layer, then presentation layer.

I want to call out something nice about this naming: **lens** carries real semantic weight here that wasn't true before. A lens *focuses light from a scene through a constructed apparatus to produce an image* — the dio is the scene, the lens is the apparatus, the scenery is the image. The metaphor isn't just decorative, it tells you what each layer does.

(One footnote: "lens" in the Haskell/FP sense — getter/setter pairs over data — is unrelated, and your construct isn't that. Worth a mention in docs so Rust users don't pattern-match wrong, but it's not a real conflict.)

What each layer owns (and doesn't)

Vista owns: the native API surface, capability declarations, error reporting. Vista does *not* retry, cache, or coordinate. It's allowed to be unreliable — that's the whole point.

Dio owns: capability composition. "I have a read-only vista A and a write-vista B; together I present a read-write surface." "I have an unsortable vista; combined with a local sort buffer, I present a sortable surface." Dio does not cache long-term or schedule — it's a structural transform over vistas. Cheap to construct.

Lens owns: everything time-shaped. Caching, scheduling, prefetch, retry, backoff, rate-limit shaping, optimistic writes with rollback, progressive in-memory sort merges, viewport tracking, subscription fanout. The lens is where async lives. The lens is where the rust thread you described actually runs (or rather, where the tasks it spawns are coordinated).

Scenery owns: nothing. It's a view. The frontend reads from it, the lens writes to it. It's not a process or an actor; it's the published state.

Communication shape (rust-native)

You said rust primitives — channels, signals, async — so let me sketch the wire protocol in those terms rather than abstract messages.

Frontend → Lens (commands, via an `mpsc::Sender<LensCommand>` or per-scenery channel):

rust

```
enum LensCommand {
    Subscribe { query: Query, scenery_tx: SceneryHandle },
    SetViewport { scenery: SceneryId, range: Range<usize> },
    Refresh { scenery: SceneryId },
    Edit { id: RecordId, patch: Patch },
    Insert { record: Record },
    Delete { id: RecordId },
    Unsubscribe { scenery: SceneryId },
}
```

Commands are fire-and-forget. The lens always responds via state updates on the scenery channel, never via reply. This keeps the model clean: the frontend never `await`s a command result, it reacts to scenery changes.

Lens → Scenery (a `watch::Receiver<SceneryState>` plus a `mpsc::Receiver<Delta>`):

There's a natural split here:

- A **watch channel** carries the latest `SceneryState` — query status, loading flags, error state, viewport echo. The frontend reads this for "the current overall situation." Coalescing is desired (frontend only needs the latest).
- A **delta stream** (mpsc or broadcast) carries row-level events: `Patch(id, data)`, `Insert(pos, row)`, `Remove(id)`, `RangeReplaced(range, rows)`, `Invalidated`. These must not coalesce; ordering matters.

The frontend's table widget consumes both: the watch channel drives the chrome (spinner, error banner, "synced 3s ago"), and the delta stream drives the row model.

Lens internals:

A lens is a long-lived task (or set of tasks) holding:

- The two-layer cache (row store + query index)
- A pending-writes log (optimistic patches not yet confirmed)
- A scheduler (probably a priority queue of pending vista operations, with viewport-derived priority)

- A set of active sceneries (each with a query, viewport, output channels)
- A handle to the dio it's focusing

Internal signaling between these is also channels and `Notify`. The scheduler wakes on: command arrival, vista response, retry timer fire, rate-limit budget refill, optimistic write timeout.

How a few scenarios play out concretely

Scenario A: frontend subscribes to "all cats," vista is paginated and slow.

1. Frontend sends `Subscribe { query: Query::filter("species", "cat"), scenery_tx }`.
2. Lens creates a Scenery record, stores the channels, sets `status.loading = true`, publishes initial state.
3. Lens checks cache for query results. Cache empty. Lens asks dio: "fetch page 1 of cats." Dio may translate this into "fetch page 1 from vista, filter locally" if vista can't filter natively.
4. Page 1 returns. Lens writes rows into the row cache, appends ids to the query index, emits `RangeReplaced(0..n, rows)` on the delta stream, updates `status` (`loading = false` if all pages done; otherwise stays loading with progress field).
5. Lens immediately schedules page 2. While that's in flight, frontend can already render page 1.
6. User scrolls. Frontend sends `SetViewport { range: 200..250 }`. Lens checks query index — page 5 not yet fetched. Lens bumps page 5 to top of scheduler queue. Pages 2-4 still get fetched, just lower priority.

Scenario B: refresh.

1. Frontend sends `Refresh`.
2. Lens marks all query-index entries for this scenery as "refreshing" (status update fires). Existing rows stay visible but get `Stale` tag in their status field on the row cache (a single `Patch` event per row, or batched).
3. Lens re-fetches page 1. Compares incoming rows to cached rows. For each:
 - Same id, same data: no event.
 - Same id, different data: `Patch(id, new_data)`.
 - New id: insert into query index, emit `Insert(pos, row)`.
 - Cached id absent from new page: emit `Remove(id)`.
4. Continues through pages, emitting events as it diffs.
5. When done: `status.refreshing = false`, all rows fresh again.

The diffing logic lives in the lens, not the driver. Driver just returns pages.

Scenario C: user edits row, network is dead.

1. Frontend sends `Edit { id: 47, patch: {...} }`.
2. Lens appends to pending-writes log. Applies patch optimistically to the row cache (this is the read-your-writes guarantee). Emits `Patch(47, optimistic_data)` and a status change on row 47 (`status = PendingWrite`).
3. Lens schedules the write against the vista. Vista returns network error. Retry policy kicks in — exponential backoff.
4. Meanwhile, user can keep editing. Each edit either replaces the pending write (if not yet sent) or queues a follow-up.
5. Network comes back. Write succeeds. Pending-writes log clears the entry. Row status returns to `Fresh`. Optimistic data is now confirmed data; no event needed if data matches what the lens already showed.
6. Alternative: write fails permanently (validation error from backend). Row status → `WriteFailed { error }`. Lens emits a status update; frontend shows error UI; user can retry or revert. Cache row remains optimistic until user decides.

Scenario D: progressive sort.

The frontend wants "all cats sorted by age desc" but the vista can't sort. Dio composition: sortable-dio wraps unsortable-vista. The dio's contract says "I can sort, but I need to see all data to do it."

Lens semantics: query is "all cats sorted by age desc." Lens fetches pages. After each page arrives, lens merges into a locally-sorted result, emits a `Invalidated` followed by `RangeReplaced(0..total_so_far, sorted_rows)`. Or, more elegantly, emits patches/inserts/removes that transform the previous sort into the new one — a real diff.

The `status` watch channel exposes `fetches: 340, estimated_total: ~2000`, so the frontend can render "Sorting... 17%".

If the user scrolls to row 1000 during this: that's fine, row 1000 doesn't exist yet in the sorted view. Status carries `total_known: 340`, frontend's virtual scroller respects that and snaps back or shows "loading more results."

Questions to settle before going deeper

1. **Single lens or lens-per-table?** I've been writing as if a lens corresponds to one logical table. But you might want one lens per process managing many tables. I'd argue lens-per-table is cleaner — the cache, scheduler, and rate-limit policies are usually per-backend-table anyway. Confirm?
2. **Is dio composition declarative or imperative?** Declarative would be a builder: `Dio::new().with_reader(csv).with_writer(memory).with_sort_in_memory().build()`. Imperative would be hand-written dio impls per use case. Probably you

want declarative for the common cases and a trait so people can hand-roll exotic ones.

3. **Do sceneries share caches across lenses?** I assumed not — each lens owns its cache. But if two lenses point at the same underlying vista, the row cache *could* be shared. I'd say no: lenses are independent, and if you want sharing, you share the dio (and the vistas it composes) but not the lens. Caches are lens-local.
4. **How does the lens learn about external changes?** If the backend supports change notifications (CDC, websockets, polling), is that the lens's job or the vista's? I'd say the *vista* exposes an optional "changes" stream as a capability; if present, the lens subscribes to it and emits the resulting patches. If absent, refresh is the only way.
5. **Cancellation.** When a scenery unsubscribes mid-fetch, do in-flight vista calls cancel? Probably yes for expensive ones, no for cheap ones (the result still goes in the cache and might benefit a future scenery). The lens's scheduler decides.

The thing I keep coming back to is that **the lens is doing 90% of the work**, and the driver author's job is mostly to implement vistas well and pick the right dio composition. That feels right — it matches your goal of "developer implements the gaps." Vista capabilities + dio composition declare the gaps; the lens fills them.

Where do you want to drill in? My suggestions in order: (a) the exact capability enum and how dios advertise composed capabilities, (b) the scheduler design inside the lens, (c) the optimistic-write protocol in detail, or (d) the rust-level types for SceneryState and the delta stream.

question - you mentioned tehre are already existing protocols for this thing. Can you walk me through some existing designs ?

Good instinct to ask. This problem space has been chewed on hard by several communities, and there are designs worth stealing from. Let me walk through the ones that map most directly onto what you're building, what each gets right, and where they fall short for your use case.

1. Apollo Client / Relay (GraphQL client caches)

These are the closest commercial analogues to your lens.

Architecture:

- **Normalized cache:** a flat `id → object` store, exactly your row cache. Every object the client has ever seen lives here keyed by its global id.
- **Query results store:** each query is a tree of *references* into the normalized cache. Asking for "all cats sorted by age" returns a list of ids; the actual row data is

resolved through the normalized cache. This is exactly your two-layer split.

- **Subscriptions:** components subscribe to queries; when any row in the normalized cache changes, the cache walks the query results that reference it and re-emits to affected subscribers.
- **Optimistic updates:** a mutation can apply an "optimistic response" immediately, which is a patch layered on top of the cache. On confirmation it's promoted; on failure it's rolled back. The cache reads always *combine* the base layer with the optimistic layer, so subscribers see optimistic data without it being committed.
- **Field-level fetch policies:** each field can specify `cache-first`, `cache-and-network`, `network-only`, etc. This is the "staleness policy per query" mechanism.

What's worth stealing:

- The id-keyed normalized store as an event bus. Single source of truth for row identity. When row 47 changes, every scenery that references id 47 gets notified, regardless of which query they came from.
- Optimistic patches as a *layer* over the cache, not a mutation of the cache. Clean rollback.
- The query → list-of-ids representation. Cheap to update structurally (insert, remove) without touching row data.

What doesn't fit:

- Apollo assumes GraphQL — a query language that already supports filter/sort/pagination natively. Your problem is the *opposite*: capability-poor backends. Apollo doesn't address "the backend can't sort, so do it locally."
- Apollo's cache lives in the browser and is small. Your cache might need to be substantial and disk-backed for "show stale data when reopening the app tomorrow."
- Apollo's mutation model assumes the server is reliable and the network is the only flake. Your model assumes the server is fundamentally eventually-consistent and capability-constrained.

Map to your design: Lens ≈ Apollo's `InMemoryCache` + `QueryManager`. Scenery ≈ a subscribed query result. Optimistic-writes log ≈ Apollo's optimistic response layer.

2. Local-first / CRDT systems (Replicache, Automerge, Yjs, Linear's sync engine)

These are the spiritual cousins of your "infallible reactive surface over fallible backend."

Architecture (Replicache is the cleanest example):

- Client holds a complete local replica of the relevant data subset.

- All reads are local. They never fail, never await network.
- All mutations are *functions* (mutators) that run locally first, then re-run on the server. The mutator output is the source of truth.
- Sync is a separate background process: pull server state, rebase local pending mutations on top.
- Subscriptions: components subscribe to *queries against the local replica*. When the replica changes (via local mutation or sync), affected subscriptions re-fire.

What's worth stealing:

- The mental model of "the lens is a local replica; the vista is the upstream truth; sync is a background process." This is exactly the inversion you want — frontend talks to a local thing, the local thing reconciles with the unreliable backend on its own time.
- Mutations as data, not as RPCs. You queue a mutation, it runs locally immediately, it persists, and eventually it reaches the server. If the network is dead for an hour, the queue fills up; when the network returns, the queue drains. The mutation is durable across app restarts.
- Sync as pull-rebase: client says "I'm at version X, give me everything since then." Server replies with a patch. Client reapplies pending local mutations on top of the new base.

What doesn't fit:

- Replicache assumes you can have a complete local replica of the user's data. For "table with 2 million rows" this doesn't work. You need partial replication driven by queries/viewport.
- CRDTs (Automerge, Yjs) assume you can do automatic conflict resolution at the data structure level. Most business data isn't a CRDT — "user changed price from £10 to £12, server changed it from £10 to £15" is a conflict requiring human judgment, not auto-merge.
- These systems assume the backend cooperates with the sync protocol. Your DynamoDB / CSV / random API does not.

Map to your design: This is the model your lens is *secretly implementing*, just without CRDT-level guarantees. The optimistic-writes log is your mutation queue. The cache is your local replica. The vista refresh logic is your pull-rebase.

The book to read here is the [local-first essay by Ink & Switch](#) — defines the seven properties, articulates why this model exists.

3. Materialized view systems (Materialize, RisingWave, Differential Dataflow)

These are the database-side cousins. You're not building one of these, but the conceptual model is identical.

Architecture:

- Queries are *registered* as views.
- The system maintains each view incrementally: when underlying tables change, deltas propagate through the view graph and update results.
- Results are streams of `(row, +1)` and `(row, -1)` — insertions and retractions.

What's worth stealing:

- The **+1/-1 delta vocabulary**. Every change to a query result is either "this row entered" or "this row left." Updates are a retraction followed by an insertion. This is more uniform than your `Patch | Insert | Remove` set and might collapse cleanly. Worth considering for the protocol.
- Differential dataflow's idea that a query is a *standing computation*, not a one-shot request. Your sceneries are standing queries. The lens is a (tiny, single-node) differential dataflow engine.

What doesn't fit:

- These systems own the entire data path. They're not adapters over capability-poor backends.
- They assume the backend pushes changes. Your model is mostly pull with optional push.

Map to your design: Sceneries are materialized views. Delta events are the +/- stream. The lens maintains them incrementally. You're building a degraded, polling-based, single-table version of this.

4. Caching reverse proxies with stale-while-revalidate (HTTP, Cloudflare, SWR/React Query)

These are the small-scale, request-oriented version of what you want.

Architecture (TanStack Query / SWR are the React-side examples):

- Each query has a key.
- Cache stores `(key → data, fetched_at, status)`.
- When a component asks for a key:
 - If cached and fresh: return cached.
 - If cached and stale: return cached *and* fire a refetch; emit again when fresh.
 - If absent: fire fetch, emit loading state, then data.

- Refetch on focus, on network reconnect, on interval. All declarative.
- Mutation hooks invalidate keys; affected queries refetch.

What's worth stealing:

- **Stale-while-revalidate is exactly what you want for "show stale data immediately, refresh in background."** Don't reinvent the state machine — it's:

```

idle → loading → success → (stale →) revalidating → success
                ↓                               ↓
                error                          error (keep stale data visible)

```

- The invalidation-by-key pattern. Mutations declare "I touched these keys" and queries matching those keys refetch. Coarse but simple.
- Refetch triggers as policy: on focus, on reconnect, on interval, on demand. These are independent policies the user can mix and match.

What doesn't fit:

- Query/SWR caches are per-query blobs. No row-level identity, no normalization. Two queries returning overlapping rows duplicate the data and can show inconsistent results. You explicitly want to avoid this.
- No optimistic updates without manual wiring per mutation. Your design wants this automatic.
- No pagination/viewport awareness built in (React Query has "infinite queries" but it's a separate API).

Map to your design: Per-query caches with TTL and refetch triggers are a piece of your lens. Just the query-index layer, not the row cache.

5. CQRS + Event Sourcing (the heavyweight enterprise pattern)

I'll touch this briefly because it's *adjacent* to your model and worth understanding to know why you're not doing it.

Architecture:

- Write side: commands produce events, events go to an event log. Append-only.
- Read side: separate read models, built by projecting events. Multiple read models, each optimized for a query pattern.
- Consistency: read side is eventually consistent with write side by construction.

Relevance: Your design has a *flavor* of this — the pending-writes log is an event log, the cache is a projection. But you're not committing to event sourcing as a foundation. The

vista is your source of truth, not an event log.

Don't steal this directly unless you find yourself wanting projections of the same data into multiple shaped caches. Probably overkill.

6. Operational transform / collaborative editing (Google Docs, Figma)

Mentioning for completeness — this is what you'd reach for if multiple users edit the same row concurrently and you need to merge their edits. You probably don't. Most table-row edits are "user A wins, user B sees the result." Skip unless your problem turns into a collaborative one.

The patterns that show up across all of these

Strip away the specifics and the recurring ideas are:

1. **Normalized row store + per-query reference lists** (Apollo, Relay, every serious client cache)
2. **Optimistic updates as a layered patch, not a cache mutation** (Apollo, Replicache, Linear)
3. **Mutations as durable queued data, not RPCs** (Replicache, every offline-first system)
4. **+/- delta vocabulary for query result changes** (differential dataflow, materialize)
5. **Stale-while-revalidate as the dominant freshness pattern** (HTTP, SWR, React Query, Apollo's cache-and-network)
6. **Subscriptions as standing queries** (Apollo, Replicache, materialized views, everything reactive)
7. **Pull-rebase sync** (Replicache, git, every modern sync engine)

Your design already contains 1, 2, 5, and 6. Worth considering adding:

- **(3) Durable mutation queue.** You have a pending-writes log but I described it as in-memory. If you want "user edits a row, closes the app, opens it tomorrow, the edit still gets sent," it needs to be on disk. This is the single biggest robustness win for the price.
- **(4) +/- as the delta primitive.** Might simplify your protocol. Probably not worth disrupting the design over, but worth considering.
- **(7) Versioned pull-rebase.** If the vista can expose a version/cursor ("everything since version 1234"), refresh becomes a delta fetch instead of a full re-page-and-diff. This is a vista capability opt-in.

What's genuinely novel about your design

Two things that aren't well-served by any existing system:

1. **Capability-typed backends with composition.** None of the above assume the backend can't sort or filter. Apollo/Relay assume GraphQL servers; Replicache assumes a cooperating mutation/pull endpoint; React Query is agnostic but doesn't try to compensate for backend gaps. Your dio concept — composing vistas to fill capability gaps — is the actual research contribution here.
2. **Local sort/filter over partial paginated data with progressive emission.** This is mentioned in materialize literature but not as a first-class client pattern. Showing the user a sort that's filling in as pages arrive is a real UX innovation, especially with the "X% complete" surfacing.

The rest of your design is well-trodden ground, which is *good* — it means you can borrow patterns confidently rather than discovering them painfully.

If you want to go deeper on any one of these, I'd suggest:

- **Replicache's design docs** for the cleanest articulation of the local-replica-plus-pull-rebase model. Their concept docs are unusually well-written.
- **Apollo's normalized cache internals** (their "Demystifying Cache Normalization" post, or the source) for the row store + query result references pattern in production.
- **Differential dataflow's papers** (Frank McSherry's blog, the original DD paper) if you want to understand the +/- delta model and incremental view maintenance properly. Mind-expanding but heavy.
- **TanStack Query's source** for a small, readable example of stale-while-revalidate + invalidation in ~10k lines.

Which of these do you want to dig into? Or shall we go back to designing the lens with these patterns in mind?

amazing, that's very useful. keep those alternative models in mind. We need to think of Dia protocol and how it is implemented. Also we should decide on a cache - which is built in rust, lightweight and can be stored in memory or written to disk transparently. I thought ReDB or redis, but if you know anything that's more suitable - i'm game.

3 things cache need to handle:

.

Show more

Good, this is taking real shape. Let me push on the cache choice first because it's load-bearing, then design the cache schema and Dia protocol around what we pick.

Cache backend — let me rule things in and out

You named ReDB and Redis. Quick reality check on both, then a wider survey:

Redis is the wrong shape for you. It's a separate process you connect to over a socket. You said "lightweight, in memory or disk transparently" and "built in rust." Redis is none of those — it's C, it's networked, and "transparently in memory or disk" isn't really how it works (persistence is configurable but it's always a server). The serialization overhead of going over a Unix socket for every cache read inside a hot reactive loop is significant. Rule it out.

ReDB is a much better fit. Pure Rust, embedded, single file, MVCC, typed tables, copy-on-write B-trees. It's small (~10k LOC), no unsafe in the public API, the author (cberner) is careful. Real downside: durability is the only mode — every transaction commits to disk by default. You can defer commits but you can't run it as a pure in-memory store cleanly. For your use case that's actually fine *if* the disk option is what you usually want.

The full survey of credible options:

- **redb** — embedded, pure Rust, typed K/V tables with `TableDefinition<K, V>`, MVCC, single writer, multiple readers. Mature enough for production. Good ergonomics.
- **sled** — embedded, pure Rust, lock-free, was popular for years. Author has been working on a v1.0 rewrite for ages and the existing 0.34 is "use at your own risk" per the README. I'd avoid for new work. Sad because the API is lovely.
- **fall** — embedded, pure Rust, LSM-tree based (RocksDB-shaped). Newer (1.0 in 2024). Multiple "partitions" (column families), tunable memtable, optional block cache. Good fit if you want LSM characteristics — write-heavy workloads, range scans over sorted keys.
- **heed** — Rust bindings over LMDB. LMDB is rock-solid C code with decades of production use. Memory-mapped, very fast reads, single-writer. Not pure Rust which violates your "built in rust" preference, but worth knowing about.
- **persy** — pure Rust embedded, transactional, less well known. Smaller community than redb.
- **SQLite via rusqlite or sqlx** — not "lightweight" by some definitions but by others it's the lightest. Embedded, battle-tested, you get SQL for free. Could be overkill, could be exactly right.
- **In-memory only options** if you want a separate hot tier: `dashmap` (concurrent hashmap), `moka` (full-featured async cache with TTL/LRU/LFU), `quick_cache`.

My recommendation: redb as the durable layer, moka as the hot tier in front of it.

Reasons:

- redb gives you typed tables, MVCC reads (great for the read-heavy lens), and a single-file format that's easy to ship and reset.
- moka gives you the "in-memory transparently" part — sub-microsecond reads on hot keys, TTL, LRU, all the cache hygiene you want. It's async-aware (`moka::future::Cache`) which fits your tokio runtime.
- They compose naturally: moka holds the working set, redb is the durable source of truth, you write through moka on writes and read-through on misses.
- Both are pure Rust and well-maintained.

If you want a single-store answer, just **redb** is defensible — its B-tree is mmap-friendly and reads are fast. You'd lose the in-memory-only operating mode but you'd avoid the consistency complexity of two tiers.

If you wanted SQL queries over your cache for ad-hoc inspection or because you have richer query shapes than K/V, **SQLite** isn't crazy. It's heavier conceptually but you'd get a real query engine. I'd default to redb unless you find yourself fighting it.

What about Sanakirja, agdb, or oxide-rs?

Briefly: niche. Sanakirja is the storage engine behind Pijul, very clever, less documented. Not worth the learning curve over redb. The others are unproven.

Cache schema

Given redb's `TableDefinition<K, V>` model, here's the schema. Three logical tables, plus metadata.

```
rust

// Row store: id → (data, metadata)
const ROWS: TableDefinition<RecordId, StoredRecord> = TableDefinition::new("rows")

struct StoredRecord {
    data: Bytes,           // serialized row, format is the dia's choice
    version: u64,          // monotonic per record, for staleness checks
    fetched_at: Timestamp,
    source_etag: Option<Bytes>, // backend's version tag if available
    status: RecordStatus,   // Fresh | Stale | PendingDelete
}
```

`RecordId` is whatever the dia declares — string, UUID, composite. Use `bincode/postcard` for serialization.

```
rust

// Sorted chunks: (query_id, chunk_index) → ordered list of ids
const CHUNKS: TableDefinition<(QueryId, u32), Chunk> = TableDefinition::new("chunks")

struct Chunk {
    ids: Vec<RecordId>,
    backend_cursor: Option<Bytes>, // cursor to fetch the NEXT chunk
    fetched_at: Timestamp,
    status: ChunkStatus,           // Fresh | Stale | PartiallyInvalidated
    sort_key_range: Option<(SortKey, SortKey)>, // first/last sort key in chunk
}
```

`QueryId` is a stable hash of the query (sort columns + filter + filter args). Chunks within a query are ordered by `chunk_index`. The `sort_key_range` field is the cleverness — it lets you locate which chunk a row *should* be in for patching, without scanning everything.

```
rust

// Query metadata: query_id → query info
const QUERIES: TableDefinition<QueryId, QueryMeta> = TableDefinition::new("queries")

struct QueryMeta {
    query: SerializedQuery,           // for re-issuing on refresh
    chunk_count: u32,                 // how many chunks fetched so far
    complete: bool,                   // has the entire result been fetched?
    total_estimated: Option<u64>,     // backend's estimate if available
    sort_keys: Vec<SortKey>,          // columns we sorted by
    last_full_refresh: Timestamp,
    next_cursor: Option<Bytes>,       // cursor for fetching the next page
}
```

```
rust

// Pending writes: ordered log of mutations not yet confirmed
const PENDING: TableDefinition<u64, PendingMutation> = TableDefinition::new("pending")

struct PendingMutation {
    sequence: u64,                    // strictly monotonic
    record_id: RecordId,
}
```



```

operation: MutationOp, // Insert | Update | Delete
payload: Bytes,
attempts: u32,
last_attempt: Option<Timestamp>,
last_error: Option<String>,
created_at: Timestamp,
}

```

This is your durable mutation queue. Critical for "user edits, closes app, edit still sends tomorrow."

How the cache handles your three scenarios

Scenario 1: dynamodb returns 300 unsorted records, we sort and store.

1. Lens issues `fetch_all` against the dia, dia returns pages (let's say 100 each).
2. After each page arrives, lens writes raw rows to `ROWS` table.
3. Lens accumulates ids in a temporary in-memory buffer, sorts as new ids arrive (incremental merge), and writes sorted chunks to `CHUNKS` table.
4. Each chunk gets `sort_key_range` filled in.
5. `QueryMeta` updated with `chunk_count`, `complete = true` when done.

The lens emits scenery updates progressively as chunks complete.

Scenario 2: change event for record 47 arrives.

1. Dia receives or polls the change event, hands lens a "record 47 changed" notification.
2. Lens looks up record 47 in `ROWS`. Compares old data with new.
3. **If sort columns unchanged:** just update `ROWS[47]`, emit `Patch(47, new_data)` to all sceneries showing it. Done — chunks unaffected.
4. **If sort columns changed:** this is the hard case.
 - Determine record 47's *old* position: which `(QueryId, chunk_index)` contains it? Two ways to know:
 - Maintain a reverse index: `(QueryId, RecordId) → chunk_index`. Extra table, extra write per chunk update, but O(1) lookup.
 - Binary search chunks using `sort_key_range` if you know the *old* sort value. The lens can read it from cache before the update.
 - Remove id 47 from old chunk.

- Compute new sort key, binary search `sort_key_range` to find target chunk.
- Insert id 47 into new chunk at the right offset.
- Update affected `sort_key_range` values.
- Emit `Remove(47)` then `Insert(new_pos, row)` to scenery — or compute the net `Move` if your protocol supports it.

I'd add a reverse index table:

```
rust
```

```
const REVERSE: TableDefinition<(QueryId, RecordId), u32> = TableDefinition::new(
// value is chunk_index
```

It costs one extra write per chunk update but turns the "where is record 47?" lookup from $O(\log \text{ chunks})$ into $O(1)$. Worth it.

Scenario 3: app restarts, 3 pages cached, rest stale.

1. Lens opens redb, reads `QueryMeta` for the active query.
2. `complete = false`, `chunk_count = 3`, `next_cursor = Some(...)`.
3. Lens marks the 3 cached chunks as `Stale` (still serve them to scenery for instant rendering, with a stale indicator).
4. Lens starts refresh from `next_cursor` to continue fetching pages 4+.
5. Meanwhile, lens decides whether to *also* re-verify pages 1-3. Policy choice — see below.

Cache policies the dia developer can choose

These are the "various parameters" you mentioned. Each is a knob the dia developer sets when implementing for a particular backend:

Freshness policy

- `cache_first` — serve cached if any, never refetch unless asked
- `stale_while_revalidate` — serve cached, refetch in background, emit updates
- `network_first` — refetch on subscribe, fall back to cache if network fails
- `cache_and_network` — emit cached immediately, also emit network result (Apollo-style)

Pre-warming policy

- `none` — fetch only when scenery asks

- `eager_on_construct` — start fetching everything when the lens is built
- `subscribed_then_fetch` — subscribe to change stream first, then fetch (your "subscribe to events and start fetching" pattern)
- `prefetch_viewport_neighbors` — when viewport is at chunk N, prefetch chunks $N \pm 2$

Restart policy

- `trust_cache` — cached data is valid on restart
- `verify_first_chunk` — re-fetch chunk 0 on restart, if same as cached then trust rest
- `mark_all_stale` — show cached data with stale indicator, refresh in background
- `discard` — wipe cache on restart (debug only)

Sort/filter policy (this is the dio composition part)

- `native` — backend can sort and filter natively, pass through
- `fetch_all_then_sort_locally` — fetch every page, sort in memory, chunk and cache
- `fetch_ids_sorted_then_hydrate` — your second example: backend returns sorted ids only, fetch row data on demand
- `hybrid` — backend sorts by primary key cheaply, lens re-sorts on secondary key for small result sets

Eviction policy

- `keep_all` — never evict (use only for small datasets)
- `lru_chunks` — evict cold chunks, keep `ROWS` longer
- `ttl` — drop chunks older than X
- `size_bounded` — total cache size cap

These policies are *declarative* on the dia. The lens reads them and behaves accordingly. The dia developer doesn't write retry loops or eviction code — they pick policies.

The Dia protocol

This is the trait surface the dia exposes to the lens. Capability-typed, with optional capabilities advertised via separate traits the dia may or may not implement.

```
rust
/// The core, mandatory surface every dia must implement.
trait Dia: Send + Sync + 'static {
```

```

type RecordId: Eq + Hash + Clone + Serialize + DeserializeOwned + Send + Sync
type Record: Serialize + DeserializeOwned + Send + Sync;
type Cursor: Serialize + DeserializeOwned + Clone + Send + Sync;
type Query: Serialize + DeserializeOwned + Clone + Hash + Send + Sync;
type Error: std::error::Error + Send + Sync;

/// Declare what this dia can do natively.
fn capabilities(&self) -> Capabilities;

/// Declare cache and refresh policies.
fn policies(&self) -> Policies;

/// Fetch one page starting at the given cursor (None = start).
/// Returns rows and the next cursor (None if done).
async fn fetch_page(
    &self,
    query: &Self::Query,
    cursor: Option<Self::Cursor>,
    limit: usize,
) -> Result<Page<Self::RecordId, Self::Record, Self::Cursor>, Self::Error>;

/// Fetch a specific record by id. Required even if pagination covers it,
/// because lens uses this for hydration and verification.
async fn fetch_one(
    &self,
    id: &Self::RecordId,
) -> Result<Option<Self::Record>, Self::Error>;
}

struct Page<Id, Rec, Cur> {
    rows: Vec<(Id, Rec)>,
    next_cursor: Option<Cur>,
    total_estimated: Option<u64>,
}

```

rust

```

/// Optional: dia can mutate.
trait DiaWrite: Dia {
    async fn insert(&self, record: Self::Record) -> Result<Self::RecordId, Self::Error>;
    async fn update(&self, id: &Self::RecordId, patch: Patch) -> Result<(), Self::Error>;
    async fn delete(&self, id: &Self::RecordId) -> Result<(), Self::Error>;
}

```

```

/// Optional: dia can sort natively.
trait DiaSort: Dia {
    fn supported_sort_keys(&self) -> &[SortKey];
}

/// Optional: dia can filter natively.
trait DiaFilter: Dia {
    fn supported_filters(&self) -> FilterCapability;
}

/// Optional: dia can stream change events.
trait DiaSubscribe: Dia {
    fn subscribe(&self) -> impl Stream<Item = ChangeEvent<Self::RecordId>> + Send;
}

enum ChangeEvent<Id> {
    Inserted(Id),
    Updated(Id),
    Deleted(Id),
    Invalidated, // wholesale; resync needed
}

/// Optional: dia can fetch many records at once.
trait DiaBatch: Dia {
    async fn fetch_many(
        &self,
        ids: &[Self::RecordId],
    ) -> Result<Vec<(Self::RecordId, Option<Self::Record>>), Self::Error>;
}

/// Optional: dia exposes a version/cursor for incremental refresh.
trait DiaVersioned: Dia {
    type Version: Serialize + DeserializeOwned + Clone + Send + Sync;
    async fn fetch_changes_since(
        &self,
        version: &Self::Version,
    ) -> Result<ChangeBatch<Self::RecordId, Self::Record, Self::Version>, Self::Error>;
}

```

The lens uses these capabilities to plan. Examples:

- Refresh: if `DiaVersioned`, fetch incremental delta. Else if `DiaSubscribe`, just trust the subscription. Else, page through everything and diff.

- Sort: if `DiaSort` declares the key, push down. Else, fetch all and sort locally (requires `policies.sort = fetch_all_then_sort_locally`).
- Optimistic write: requires `DiaWrite`. Lens handles retry, durability, rollback regardless of what `DiaWrite` does internally.

The dia developer's job becomes: implement `Dia` (mandatory), opt into traits they can support, configure policies. The lens does the rest.

Composition (the "dio" part)

A composed dia wraps multiple inner dias. Concrete example:

```
rust

/// Combines a read-only CSV dia with an in-memory write dia.
/// Reads merge both; writes go to the writable inner; updates to read-only
/// records are written as overlays in the writable inner.
struct OverlayDia<R: Dia, W: Dia + DiaWrite> {
    base: R,
    overlay: W,
    overlay_index: RwLock<HashMap<R::RecordId, ()>>, // ids overlaid
}

impl<R, W> Dia for OverlayDia<R, W>
where R: Dia, W: Dia + DiaWrite<RecordId = R::RecordId, Record = R::Record>
{
    type RecordId = R::RecordId;
    type Record = R::Record;
    // ...

    fn capabilities(&self) -> Capabilities {
        // We are write-capable because overlay is.
        // We are sort-capable iff both base and overlay are.
        Capabilities {
            write: true,
            sort: self.base.capabilities().sort && self.overlay.capabilities().sort,
            // ...
        }
    }

    async fn fetch_page(&self, query, cursor, limit) -> Result<...> {
        // Fetch from base, replace overlaid ids with overlay versions.
        // Cursor format encodes which inner dia we're paginating through.
    }
}
```



```
impl<R, W> DiaWrite for OverlayDia<R, W> where ... { /* always writes to overlay
```

This is the pattern. Dio compositions are *just dias* — they implement `Dia` (and optionally the capability traits) by combining other dias. The lens sees no difference between a leaf dia and a composed one.

Common compositions worth shipping as library primitives:

- **OverlayDia** — read-only base + writable overlay → read-write
- **SortDia** — unsortable inner + sort-buffer → sortable (loads all into memory)
- **FilterDia** — unfilterable inner + local filter → filterable
- **CachingDia** — any inner + in-memory cache → faster + can serve while inner refetches
- **RateLimitedDia** — any inner + token bucket → rate-limit-compliant
- **RetryDia** — any inner + retry policy → retries transient errors

You stack these like middleware:

```
rust
let dia = OverlayDia::new(
    SortDia::new(
        RateLimitedDia::new(
            DynamoDia::new(table_name),
            tokens_per_sec: 10,
        ),
        sort_key: SortKey::new("age", Desc),
    ),
    MemoryWriteDia::new(),
);
```

The capability set propagates upward through composition. The lens at the top sees: read, write, sort, rate-limited (informational), retry-aware.

Where I want to push back / questions

Q1. Is the dia API id-centric or record-centric? I've made `RecordId` mandatory in fetched pages. Some backends don't have an obvious id (raw CSV rows). Options: dia declares an id extraction function, lens assigns synthetic ids, or dia returns `(Option<Id>, Record)` and the lens generates ids for the nameless. I'd lean on "dia must declare an id extractor" even if it's `row_index_in_file`.

Q2. What's the granularity of change events from `DiaSubscribe`? Row-level is ideal but many backends only have "something changed in this table." If granularity is "the whole table is dirty," the lens has to refetch everything to find changes — at which point the subscription is just a refresh trigger. Worth declaring this explicitly in `Capabilities`.

Q3. Do you want the cache to be queryable by frontend tooling? "Show me cache stats, manually evict X, inspect pending writes." If yes, we should expose an admin API. If not, keep it lens-private.

Q4. Do you envisage running multiple lenses against the same redb file (e.g. several tables in one process), or one redb file per lens? Redb supports multiple typed tables in one file, so one file per process is cleaner. But it means all lenses share write-lock contention. Probably fine for your scale — flag for later.

Q5. How big do we expect a cache to get? If it's "hundreds of thousands of rows," redb is fine. If it's "millions and we'll page out cold data," we need eviction strategy upfront. If it's "tens of thousands," we don't need redb at all and can do everything in memory with snapshot-to-disk on shutdown.

Given the cache picks ripple through everything else, I'd settle Q5 first. What scale are you actually building for?

ok, before we go into schema, Vista already creates unified data types with strong type boundaries. We have `Record<CborValue>` which we are storing. It's similar to `IndexMap`, but it can record dirty records. For instance if user opens a form but changes 1 field out of 20 - only single field will need to be patched back. I already use ReDB and it makes it a perfect candidate. I don't mind if everything is persisted on disk, even better actually. Lets also use in-memory only for kv store, with TTLs.

Table definition is of no concern either. It's already handled by Vista. No need to consider that. Focus only on data.

Good — that clarifies a lot. If Vista already gives you `Record<CborValue>` with dirty-field tracking, and the table definition / type boundaries live there, then the cache schema collapses to something much simpler. Let me redesign with that in mind.

What this changes

- No schema-per-table in the cache layer. The cache just stores `Record<CborValue>` blobs. Vista handles deserialization back into typed records on the other side.

- The dirty-field tracking is a gift. Patches over the wire are just the dirty subset; reconciliation against backend is "send only what changed."
- ReDB for everything durable, in-memory map with TTL for the hot KV tier. No moka needed if you just want a `DashMap` plus a TTL sweeper, but moka gives you LRU/LFU and async-aware eviction for free. Your call — I'd default to moka unless the dependency footprint matters.
- Cache becomes generic over `RecordId` and serialized `Record<CborValue>` bytes. No table-specific types.

The cache, redrawn

Three redb tables, period. No type parameters bleeding into the schema.

```
rust
```

```
// Durable row store. Lives on disk.
const ROWS: TableDefinition<[u8], [u8]> = TableDefinition::new("rows");
//                                     ^id                ^StoredRow (cbor-encoded)

struct StoredRow {
    record: Vec<u8>,                // CBOR-encoded Record<CborValue> from Vista
    version: u64,                   // lens-local monotonic
    fetched_at: u64,                // unix ms
    source_etag: Option<Vec<u8>>,
    status: RowStatus,              // Fresh | Stale | PendingDelete | Tombstone
}
```

```
rust
```

```
// Sorted chunks per query.
const CHUNKS: TableDefinition<([u8], u32), [u8]> = TableDefinition::new("chunk
//                                     ^query_id ^chunk_idx ^Chunk (cbor)

struct Chunk {
    ids: Vec<Vec<u8>>,              // ordered record ids
    backend_cursor: Option<Vec<u8>>, // for fetching the NEXT chunk
    fetched_at: u64,
    status: ChunkStatus,            // Fresh | Stale | PartiallyInva
    sort_key_low: Option<Vec<u8>>,  // first row's sort key
    sort_key_high: Option<Vec<u8>>, // last row's sort key
}
```

```
rust
```

```
// Query metadata.
const QUERIES: TableDefinition<&[u8], &[u8]> = TableDefinition::new("queries");
//
//          ^query_id          ^QueryMeta (cbor)

struct QueryMeta {
    query_blob: Vec<u8>,           // serialized query for re-issuing
    chunk_count: u32,
    complete: bool,
    total_estimated: Option<u64>,
    sort_keys: Vec<SortKeySpec>,  // declared sort columns
    last_full_refresh: u64,
    next_cursor: Option<Vec<u8>>,
}
```

```
rust

// Reverse index: which chunk contains record X for query Q.
const REVERSE: TableDefinition<(&[u8], &[u8])>, u32> = TableDefinition::new("reverse");
//
//          ^query_id ^id          ^chunk_idx
```

```
rust

// Pending writes log. Strictly ordered by sequence.
const PENDING: TableDefinition<u64, &[u8]> = TableDefinition::new("pending");

struct PendingMutation {
    sequence: u64,
    record_id: Vec<u8>,
    operation: MutationOp,           // Insert | UpdatePatch | Delete
    dirty_fields: Vec<u8>,           // CBOR-encoded sparse Record (Vista's patch)
    attempts: u32,
    last_attempt: Option<u64>,
    last_error: Option<String>,
    created_at: u64,
}
```

That's it. Everything stored is opaque CBOR blobs — Vista decodes them on the way out.

The in-memory hot tier

A single `Cache<Vec<u8>, Arc<StoredRow>>` from moka, keyed by record id. TTL on entries, LRU eviction, async-friendly.

```
rust

struct HotKv {
    inner: moka::future::Cache<Vec<u8>, Arc<StoredRow>>,
}
```

Read path: hot tier first, fall back to redb on miss, populate hot tier on read.

Write path: write redb first (durability), then update hot tier. If durability isn't critical for a particular write — say, a refresh that's just refilling cache — you can lazy-flush. But the simple invariant is "redb is truth, hot tier is a view."

The `Arc<StoredRow>` lets sceneries hold rows without copying. Sceneries reading from cache get cheap clones of the Arc.

How dirty-field tracking plays into the protocol

This is where Vista's design pays off concretely:

Optimistic edit flow:

1. Frontend calls `scenery.edit(id, patch)`. The patch is a `Record<CborValue>` with only the changed fields marked dirty.
2. Lens reads current `StoredRow` for that id from hot tier or redb.
3. Lens applies the patch to a clone — Vista's `Record` knows how to merge a dirty-marked patch into a full record.
4. Lens writes the merged row to redb (`ROWS[id]` updated, version bumped, status = `PendingWrite`).
5. Lens appends a `PendingMutation` to the pending log. **Crucially, it stores only the dirty-field patch, not the merged row.** This is the durable write payload.
6. Hot tier updated.
7. Lens emits `Patch(id, merged_record)` to all sceneries referencing this row.
8. Lens schedules the write against the dia. Dia's `update(id, patch)` receives only the dirty fields.

Why this matters: when the network is dead for hours and the user makes 8 edits to row 47, the pending log accumulates 8 patches OR you coalesce them into one. Coalescing dirty-field patches is *trivial* because the format is sparse — just merge the dirty-marked fields, last write wins per field. Vista's data model makes this a one-liner.

Coalescing strategy: when appending a new pending mutation, check if there's an outstanding pending mutation for the same id. If yes and both are `UpdatePatch`, merge the dirty fields and keep one entry. If the existing is `Insert` and new is `UpdatePatch`, merge into the insert payload. If existing is `Delete`, reject the update (or replace with the new operation depending on policy).

rust

```
fn coalesce_pending(existing: PendingMutation, new: PendingMutation) -> Vec<Pend
    match (existing.operation, new.operation) {
        (UpdatePatch, UpdatePatch) => vec![merge_patches(existing, new)],
        (Insert, UpdatePatch)      => vec![apply_patch_to_insert(existing, new)]
        (Insert, Delete)           => vec![], // never even send
        (UpdatePatch, Delete)      => vec![new], // drop the update
        (Delete, _)                => vec![existing, new], // unusual; keep both
        // ...
    }
}
```

This is one of the nicer wins of having dirty-field semantics in your record type. Most sync systems can't do field-level coalescing because they only have whole-record patches.

The chunk patch operation, revisited

Now that records are opaque CBOR with declared sort columns at the query level, "did the sort column change?" is a clean check:

rust

```
fn classify_row_change(
    old: &StoredRow,
    new: &StoredRow,
    sort_keys: &[SortKeySpec],
) -> ChangeClass {
    if sort_keys.iter().all(|k| extract(old, k) == extract(new, k)) {
        ChangeClass::DataOnly
    } else {
        ChangeClass::SortKeyChanged {
            old_sort: extract_all(old, sort_keys),
            new_sort: extract_all(new, sort_keys),
        }
    }
}
```

`extract` pulls a field by name from the CBOR record. Vista presumably already has this. The lens uses it for change classification.

For each active query whose `sort_keys` intersect the changed fields:

- `DataOnly`: emit `Patch(id, new_record)`, chunks untouched.

- **SortKeyChanged**:
 1. Look up old position via `REVERSE[(query_id, id)]` → old chunk_idx.
 2. Remove id from old chunk, update `sort_key_low/high` if it was a boundary.
 3. Binary search chunks by `sort_key_high` for the new position — find target chunk_idx.
 4. Insert id into target chunk at correct offset (linear scan within chunk; chunks should stay small enough — say 50-200 ids — that this is fine).
 5. Update `REVERSE[(query_id, id)]` to new chunk_idx.
 6. Emit `Move(id, old_global_pos, new_global_pos)` to scenery, or `Remove + Insert` if you don't want a Move primitive.

Chunks that don't contain the row and aren't on the boundary don't need touching. This is exactly what the chunk structure buys you.

A subtle case: what if the new sort position falls between two existing chunks (e.g. between chunk 4's `sort_key_high` and chunk 5's `sort_key_low`)? Two options:

- Append to chunk 4 (or prepend to chunk 5). Chunks can be slightly uneven in size; rebalance lazily.
- Reject the patch and mark chunks 4-5 as `PartiallyInvalid`, forcing a re-fetch of that range.

I'd go with the first — it keeps the cache useful without forcing network round trips. Periodic background rebalancing can fix size skew.

What dia gives the lens, concretely

Given the Vista context, here's the slimmed-down `Dia` trait. Generic over Vista's record id and record types — but the cache doesn't see them, only the lens-internal serialization layer does.

```
rust

trait Dia: Send + Sync + 'static {
    type Id;           // Vista's record id type
    type Query;        // Vista's query type
    type Cursor: Serialize + DeserializeOwned + Clone + Send + Sync;
    type Error: std::error::Error + Send + Sync;

    fn capabilities(&self) -> Capabilities;
    fn policies(&self) -> Policies;

    async fn fetch_page(
        &self,
```



```

        query: &Self::Query,
        cursor: Option<Self::Cursor>,
        limit: usize,
    ) -> Result<Page<Self::Id>, Self::Error>;

    async fn fetch_one(&self, id: &Self::Id) -> Result<Option<Record<CborValue>>>
    {

struct Page<Id> {
    rows: Vec<(Id, Record<CborValue>)>,
    next_cursor: Option<Vec<u8>>,
    total_estimated: Option<u64>,
}

```

`Record<CborValue>` comes back from the dia ready to serialize into `ROWS`. The lens never decodes the inner CBOR — it just stores the blob and ships it to scenery.

Patches received from the dia (via `DiaSubscribe::subscribe`) are also `Record<CborValue>` with dirty fields. The lens merges them into the cached row using Vista's merge semantics. The dirty-field representation flows end-to-end: backend change event → dia → lens cache → scenery → frontend renderer, never losing the sparseness.

Scale question, still open

The cache schema doesn't depend much on scale, but eviction policy does. For "tens of thousands of rows total" you can leave everything in redb forever and the hot tier just memoizes. For "hundreds of thousands or more" you need a real eviction policy on `ROWS` and the hot tier becomes more important.

I'd suggest:

- Hot tier: capped size (config), TTL (config, default 5 minutes), LRU.
- `ROWS` table: no eviction by default. Add a background "vacuum" task that drops rows not referenced by any `CHUNKS` entry and older than N days. Run hourly or on shutdown.
- `CHUNKS` and `QUERIES`: evict whole queries that haven't been observed by any scenery in X time.
- `PENDING`: never evict automatically. Failed mutations require user intervention.

This is policy, not mechanism — the mechanism is "redb stores it, hot tier caches it." Eviction is a periodic job that deletes rows.

Where I think you should push next

We've got:

- Cache schema (above)
- Hot tier design (moka or hand-rolled DashMap+TTL)
- Pending writes durability with field-level coalescing
- Change classification (sort-affecting vs data-only)
- Chunk patching algorithm

Things still to nail before implementation:

1. **The Capabilities and Policies enums** — exact shape, with the matrix of which capabilities the lens uses and what changes if they're absent.
2. **The lens's scheduler** — priority queue, viewport-derived priorities, rate-limit interaction, cancellation.
3. **The scenery protocol on the wire** — exact channel types, command set, delta set, watch state shape.
4. **The composition primitives (the "dios" as library)** — OverlayDia, SortDia, FilterDia, etc., with their capability inference rules.

Of these, (1) is foundational because everything else conditions on it, and (4) is where Vista users will actually do their work. Want me to write out the Capabilities/Policies enums next, or jump to one of the others?

